

This is one of the most important aspects of frontend development. The goal **isn't to make your website fit every screen size individually**. Instead, you want to build a **fluid, responsive layout** that naturally adapts to any viewport.

Since you're using **React + Material UI (MUI)**, you already have excellent tools for this. I'll explain how professional front-end developers approach it.

## 1. Think in terms of Responsive Design, not Screen Sizes

A common beginner mistake is thinking:

"I need to support 1920x1080, 1366x768, 2560x1440, 3840x2160..."

Don't.

Instead think:

- Small phones
- Large phones
- Tablets
- Small laptops
- Large monitors
- Ultrawide monitors

Everything in between should automatically adapt.

## 2. Use Fluid Layouts

Avoid fixed widths whenever possible.

Bad:

JavaScript

```
<Box width={1200}>>
```

Good:

JavaScript

```
<Box
  sx={{
    width: "100%",
    maxWidth: 1200,
    mx: "auto",
  }}
>
```

Here:

- width grows with screen
- never exceeds 1200px
- centered automatically

This alone makes a huge difference.

### 3. Use MUI Breakpoints Everywhere

MUI has built-in breakpoints.

| Breakpoint | Width  |
|------------|--------|
| xs         | 0      |
| sm         | 600px  |
| md         | 900px  |
| lg         | 1200px |
| xl         | 1536px |

Instead of

JavaScript

```
<Box width={500}>
```

do

JavaScript

```
<Box
  sx={{
    width: {
      xs: "100%",
      sm: "80%",
      md: "60%",
      lg: "50%",
    }
  }}
>
```

Each screen gets its own layout.

## 4. Never Hardcode Heights

Avoid

JavaScript

```
height: 500
```

Instead

JavaScript

```
minHeight: "100vh"
```

or

JavaScript

```
height: "auto"
```

Fixed heights often cause overflowing content.

## 5. Prefer Flexbox

Most layouts don't even need CSS Grid.

Example:

JavaScript

```

<Box
  sx={{
    display: "flex",
    flexDirection: {
      xs: "column",
      md: "row"
    },
    gap: 2
  }}
>

```

Phone:

```

Item
Item
Item

```

Desktop:

```

Item Item Item

```

No media queries needed.

## 6. Use Grid for Dashboards

Since you're making FinTrack, dashboards benefit from Grid.

Example:

JavaScript

```

<Grid container spacing={2}>
  <Grid size={{ xs: 12, md: 6 }}>
    <Paper>Chart</Paper>
  </Grid>

  <Grid size={{ xs: 12, md: 6 }}>
    <Paper>Budget</Paper>
  </Grid>
</Grid>

```

Phone

```

Chart

```

```

Budget

```

Desktop

Chart | Budget

## 7. Use Relative Units

Instead of

CSS

```
font-size: 16px;
```

use

JavaScript

```
fontSize: "1rem"
```

or

JavaScript

```
fontSize: {  
  xs: "0.9rem",  
  md: "1rem",  
  lg: "1.1rem"  
}
```

The same applies to spacing.

Instead of

JavaScript

```
padding: 20
```

use

JavaScript

```
p: 2
```

MUI spacing scales consistently.

## 8. Images Should Scale

Never

JavaScript

```
<img width={800} />
```

Instead

JavaScript

```
<img  
  style={{  
    width: "100%",  
    height: "auto"  
  }}  
>
```

## 9. Tables Need Special Handling

Tables are difficult on phones.

Instead of squeezing columns,

wrap them.

JavaScript

```
<TableContainer  
  sx={{  
    overflowX: "auto"  
  }}  
>
```

Users can scroll horizontally if necessary.

## 10. Use Container

Instead of manually centering everything,

use

JavaScript

```
<Container maxWidth="lg">
```

It automatically adjusts widths.

Phone



Laptop



4K Monitor



No giant stretched layouts.

## 11. Limit Maximum Width

One mistake I often see:

```
-----  
This line of text stretches all the way across a 4K monitor...  
-----
```

That's hard to read.

Instead

JavaScript

```
maxWidth: 1400  
mx: "auto"
```

Even on a 4K monitor:

```
-----  
Nice readable content  
-----
```

## 12. Responsive Typography

Example

JavaScript

```
<Typography
  sx={{
    fontSize: {
      xs: "1.5rem",
      md: "2rem",
      lg: "2.5rem"
    }
  }}
>
```

Large monitors get larger headings.

Phones get smaller headings.

## 13. Responsive Spacing

Instead of

JavaScript

```
mt: 10
```

use

JavaScript

```
mt: {
  xs: 2,
  md: 4,
  lg: 6
}
```

Everything breathes better.

## 14. Responsive Drawer

Instead of always showing a sidebar:

Desktop

Sidebar | Content

Phone





Content

Use

JavaScript

```
const isMobile = useMediaQuery(theme.breakpoints.down("md"));
```

Then conditionally render

- Drawer
- Permanent sidebar

## 15. Test Often

Chrome DevTools has device emulation.

Test at least:

- 360 px
- 390 px
- 768 px
- 1024 px
- 1366 px
- 1440 px
- 1920 px

Also test by gradually resizing the browser, not just using presets.

This reveals layouts that break between standard breakpoints.

## 16. Avoid Overflow

One of the biggest causes:

JavaScript

```
width: 1000
```

on a phone.

Instead

JavaScript

```
maxWidth: "100%"
```

or

JavaScript

```
width: "100%"
```

## 17. Don't Overuse `useMediaQuery`

Many beginners write

JavaScript

```
if (mobile)
  return ...
else
  return ...
```

for everything.

Instead leverage the `sx` prop:

JavaScript

```
<Box
  sx={{
    flexDirection: {
      xs: "column",
      md: "row"
    }
  }}
>
```

The browser handles the layout without duplicating components.

Reserve `useMediaQuery` for when behavior changes (such as opening a mobile drawer, disabling animations, or conditionally rendering expensive components), not just for styling.

## 18. Use CSS Functions

These are incredibly useful.

## Example

JavaScript

```
width: "min(100%, 1200px)"
```

or

JavaScript

```
width: "clamp(300px, 80vw, 1200px)"
```

These let the browser smoothly adapt sizes without relying solely on breakpoints.

## 19. Make Components Responsive, Not Pages

Design each component to be flexible on its own. For example, a `TransactionCard` should look good whether it's inside a dashboard, a modal, or a mobile view. This makes your UI much easier to maintain and reuse.

## 20. A Professional Layout Pattern

A layout like this works well for most applications, including dashboards such as FinTrack:

JavaScript

```

<Container maxWidth="xl">
  <Box
    sx={{
      display: "flex",
      flexDirection: {
        xs: "column",
        lg: "row"
      },
      gap: 3,
    }}
  >
    <Sidebar />

    <Box
      sx={{
        flex: 1,
        minWidth: 0,
      }}
    >
      { /* Content */ }
    </Box>
  </Box>
</Container>

```

#### Key details:

- `Container` keeps content from becoming too wide.
- `flexDirection` changes automatically between mobile and desktop.
- `gap` provides consistent spacing.
- `flex: 1` lets the main content fill available space.
- `minWidth: 0` is important—it allows flex items to shrink properly and prevents long content (like tables or charts) from forcing horizontal overflow.

## General Best Practices

- Use percentage-based or flexible widths ( `width: "100%"` , `maxWidth` ) instead of fixed pixel widths.
- Build layouts with **Flexbox** and **Grid**, not absolute positioning.
- Use MUI's breakpoint system ( `xs` , `sm` , `md` , `lg` , `xl` ) for layout adjustments.
- Favor responsive values in the `sx` prop over extensive `useMediaQuery` usage.
- Constrain content with `Container` or `maxWidth` for readability on large screens.

- Allow tables and other wide elements to scroll horizontally on small screens.
- Test continuously by resizing the browser and using mobile device emulation.
- Build responsive components from the start instead of retrofitting responsiveness later.

If you follow these principles, your React/MUI application will adapt gracefully from small mobile phones to ultrawide desktop monitors with minimal custom media queries.